

【专栏一：AI基础08】 - 【一张图讲清楚：RAG的原理（从“查资料”到“生成答案”全过程）】



快乐飞猪ing  · 码龄7年 · 粉丝可见 · 编辑  墨衍会员 · AI 创作全网分发

原创 · 2026-03-31 01:46:04 · 375 阅读  AI 写同款 · AI 选题 

如何通过Ragas对RAG应用进行评估：原理、指标与实战案例

Ragas是专为RAG（检索增强生成）系统设计的自动化评估框架，通过结合大型语言模型（LLM）推理...

[阅读详情 >](#)

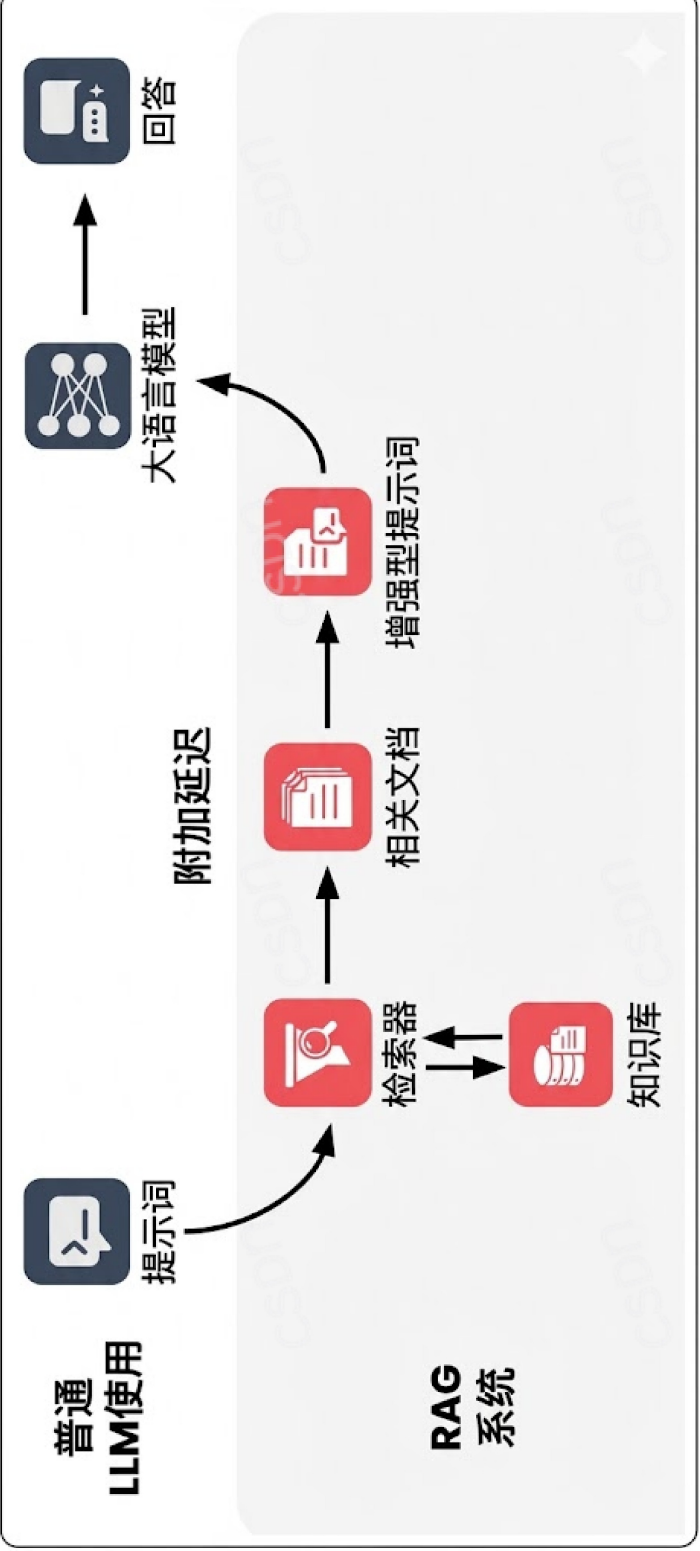
内容来源: csdn.net

作者昵称: 快乐飞猪ing

原文链接: https://blog.csdn.net/m0_46279060/article/details/159653264

作者主页: https://blog.csdn.net/m0_46279060

高级 RAG 系统架构
(Advanced RAG System Architecture)



RAG 本质上就是：先查资料，再回答。

假设你在一家公司的内部办公系统里，接入了一个 AI 助手。员工会直接问它问题，比如：

内容来源: csdn.net
作者昵称: 快乐飞猪ing
原文链接: https://blog.csdn.net/m0_46279060/article/details/159653264

- “我们公司的年假有几天？怎么申请？”

这时候，AI 助手要怎么回答？如果它只是一个普通大模型 [🔗](#)，它可能会这样：

- 按自己训练时学到的一般常识回答
- 猜一个“多数公司”的规则
- 甚至编一个看起来很像真的答案

但问题是：

- 每家公司制度都不一样，真正准确的答案只在你们公司的制度文档里。

这时候，RAG 就派上用场了。

一、一句话先讲清 RAG

在这个案例里，RAG 做的事情其实很简单：

员工提问 → **系统去知识库查制度文档** → **找到相关内容** → **把内容一起交给大模型** → **大模型基于资料回答**

也就是说，AI 助手不再是“靠记忆回答”，而是“查完资料再回答”。

第一步：先把公司的资料准备好

在员工提问之前，系统就已经在后台做准备了。

比如公司有很多文档：

内容来源: [csdn.net](https://blog.csdn.net/m0_46279060)
作者昵称: 快乐飞猪ing
原文链接: https://blog.csdn.net/m0_46279060/article/details/159653264
作者主页: https://blog.csdn.net/m0_46279060

- 员工手册
- 考勤制度
- 年假制度
- 报销流程
- 加班规则

这些文档本来只是 PDF、Word、网页之**类 Q** 的文件，模型并不能直接高效地查。所以系统第一步要做的是：
把这些文档整理成“方便检索”的形式。

具体怎么做？

比如《员工休假制度》这份文档有 20 页。系统不会整份整份去查，而是会把它拆成很多小段。例如拆成：

- 第 1 段：年假天数规则
- 第 2 段：试用期是否享受年假
- 第 3 段：请假申请流程
- 第 4 段：补休规则
- 第 5 段：法定假日说明

你可以把它想象成：本来是一整本手册，现在被拆成很多“知识卡片”。

这样做有个好处：

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159653264

作者主页：https://blog.csdn.net/m0_46279060

当用户问“年假有几天”时，系统就不用把整本手册都塞给模型，只需要找到最相关的那几张“知识卡片”就行。

第二步：把每张“知识卡片”变成可搜索的形式

拆完之后还不够，因为系统还得知道：哪些内容和“年假有几天”这个问题最相关？

这里就要用到 RAG 里很核心的一步：**向量化 (Embedding)**

这个词听起来有点吓人，但在我们的案例里，你可以这样理解：系统会把每一段文字都变成一个“语义坐标”。

比如：

- “年假天数”和“休假几天”在语义上很近
- “报销审批流程”和“年假申请”就没那么近

也就是说，系统不只是看字面关键词，而是在比较：

- 这段话的“意思”，和用户问题的“意思”像不像。

你可以把它想象成一个图书馆管理员，不是只看标题，而是大概知道每本书在讲什么。

第三步：员工提问时，系统先不急着回答，而是先去“查”

现在，真正的问题来了：“我们公司的年假有几天？怎么申请？”

内容来源: csdn.net

作者昵称: 快乐飞猪ing

原文链接: https://blog.csdn.net/m0_46279060/article/details/159653264

作者主页: https://blog.csdn.net/m0_46279060

普通大模型 Q 可能会立刻生成答案，但 RAG 系统不会，它会先做一件更稳妥的事：先把用户的问题也做一次

向量化，然后去知识库找最相关的内容。

在这个案例里，系统可能会从几百段公司制度里，先筛出这些内容：

- 《员工休假制度》第 3 页：正式员工每年享有 5 天年假
- 《员工休假制度》第 4 页：年假申请需在 OA 系统提交
- 《员工手册》第 12 页：年假需由直属主管审批

你可以把这个过程想象成：

员工问了一个问题，AI 助手没急着回答，而是先冲进公司制度库里，翻出最相关的三页资料，摊在桌上。

这就是 RAG 里面的“检索”。

第四步：不是把所有查到的内容都扔进去，而是再筛一遍

真实系统通常不会只查一次就结束。因为第一次检索到的内容里，可能有些非常相关，有些只是“沾边”。

比如系统可能还顺手查到了：

- 调休制度
- 病假制度
- 节假日安排

这些和“年假”有关，但不是这次问题的重点。

内容来源：csdn.net
作者昵称：快乐飞猪ing
原文链接：https://blog.csdn.net/m0_46279060/article/details/159653264
作者主页：https://blog.csdn.net/m0_46279060

所以更高级一点的 RAG 系统会再做一步：**重新排序 (Rerank)**

也就是再判断一次：

- **哪几段最值得给模型看**
- **哪几段可以先丢掉**

最后可能只保留最核心的 2~3 段内容。

这一步你可以理解成：**图书馆管理员先抱来一摞书，然后又快速翻了一遍，只挑出最关键的几页给你。**

第五步：把“问题 + 查到的资料”一起交给大模型

现在，系统已经拿到了最相关的资料，接下来不是直接把文档原文甩给用户，而是：**把用户问题和检索出来的资料，一起打包成一个新的提示词，再交给大模型。**

在这个案例里，模型真正收到的内容，可能是这样的：

你是公司内部制度助手，请严格根据以下资料回答问题。

- **资料 1：正式员工每年享有 5 天年假。**
- **资料 2：年假申请需通过 OA 系统提交，并由直属主管审批。**

用户问题：我们公司的年假有几天？怎么申请？这时候，大模型就不是在“猜”，而是在“**看着资料回答**”。

所以它最后生成的回答就会变成：

内容来源: csdn.net

作者昵称: 快乐飞猪ing

原文链接: https://blog.csdn.net/m0_46279060/article/details/159653264

作者主页: https://blog.csdn.net/m0_46279060

这和普通大模型最大的区别就在这里：

- 普通大模型：靠记忆、靠概率生成
- RAG 系统：靠检索到的资料增强回答

这时候，你就能真正理解 RAG 了

我们把整个案例再压缩一遍：

——用户视角——

员工问：“我们公司的年假有几天？怎么申请？”系统内部发生的事：

- 去公司知识库里找相关制度
- 找到“年假天数”和“申请流程”两段内容
- 把这两段内容和用户问题一起交给大模型
- 大模型基于资料生成答案
- 最终结果

AI 助手不再瞎猜，而是“有依据地回答”。

内容来源: csdn.net
作者昵称: 快乐飞翔ing
原文链接: https://blog.csdn.net/m0_46279060/article/details/159653264
作者主页: https://blog.csdn.net/m0_46279060

为什么 RAG 很重要？

因为现实中很多问题，答案并不在模型参数里，而在外部资料里。

比如：

- 公司内部制度
- 客服知识库
- 最新产品文档
- 法律法规更新
- 医疗指南
- 学校通知
- 项目 Wiki

这些内容有几个特点：

- 经常更新
- 属于私有知识
- 不可能每次都重新训练大模型

所以最现实的做法不是“把所有东西都塞进模型脑子里”，而是：**让模型在回答前，先去查资料。**

这就是 RAG 的价值。

二、RAG 和普通搜索有什么区别？

有些人看到这里会问：这不就是搜索吗？不完全是。

普通搜索通常只做到：

- 帮你找资料

但 RAG 会继续往后走一步：

- 帮你找资料
- 帮你读资料
- 帮你整理成最终答案

也就是说：搜索负责“找到”，RAG 负责“找到并回答”。

在我们的案例里：

- 搜索：返回《员工休假制度》第 3 页和第 4 页
- RAG：直接告诉你“5天年假，需要通过 OA 提交并由直属主管审批”
- RAG 也不是万能的

讲到这里，也要顺手提醒一句：**RAG 不是“查了资料就一定完美”**。

如果下面这些环节做不好，回答质量依然会差：

内容来源：[csdn.net](https://blog.csdn.net/m0_46279060/article/details/159653264)

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159653264

作者主页：https://blog.csdn.net/m0_46279060

- 文档切分不合理
- 检索结果不准
- 相关资料没被召回
- 提示词构造不好
- 上下文太长被截断

所以你可以把 RAG 理解为：一个“让模型更少胡说”的系统方法，而不是百分之百准确的魔法。

RAG 的本质，不是让模型更聪明，而是让模型回答前先查资料。

思考

如果 RAG 的效果不好，到底是哪一环出了问题？
是文档切分不对、检索不准，还是提示词没写好？

内容来源: csdn.net
作者昵称: 快乐飞猪ing
原文链接: https://blog.csdn.net/m0_46279060/article/details/159653264
作者主页: https://blog.csdn.net/m0_46279060